

OS_project01_10831

2022083409 컴퓨터소프트웨어학부 김승관

Design

- kernel/proc.c void scheduler(void)
 - xv6는 scheduler에서 무한루프를 돌며 proc table에 RUNNABLE한 process가 있는지 찾는다.
 - RUNNABLE한 process p를 찾으면 RUNNING 상태로 바꾸고 swtch() 함수를 사용해 현재 cpu를 점유하고 있는 process와 선택된 p를 context switch한다.
 - 다른 process가 yield()를 호출해 cpu를 포기하면 scheduler()로 돌아와 해당 process의 lock을 해제한 뒤 다시 proc table을 돌며 RUNNABLE한 process를 찾는다.

```
void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();

    c->proc = 0;
    for(;;){
        // The most recent process to run may have had interrupts
        // turned off; enable them to avoid a deadlock if all
        // processes are waiting.
        intr_on();

        int found = 0;
        for(p = proc; p < &proc[NPROC]; p++) {
            acquire(&p->lock);
            if(p->state == RUNNABLE) {
                // Switch to chosen process. It is the process's job
                // to release its lock and then reacquire it
                // before jumping back to us.
                p->state = RUNNING;
                c->proc = p;
                swtch(&c->context, &p->context);

                // Process is done running for now.
                // It should have changed its p->state before coming back.
                c->proc = 0;
                found = 1;
            }
            release(&p->lock);
        }

        if(found == 0) {
            // nothing to run; stop running on this core until an interrupt.
            intr_on();
            asm volatile("wfi");
        }
    }
}
```

Design

- kernel/proc.c void yield(void)

현재 proces의 state를 RUNNABLE로 바꾼 후 sched() 함수를 통해 CPU를 포기하게 한다.

- kernel/trap.c void kerneltrap()

which_dev == 2 즉, timer interrupt가 발생하면 yield()를 실행 해 현재 실행 중인 process가 CPU를 포기하게 한다.

```
// Give up the CPU for one scheduling round.
void
yield(void)
{
    struct proc *p = myproc();
    acquire(&p->lock);

    p->state = RUNNABLE;

    sched();
    release(&p->lock);
}

// interrupts and exceptions from kernel
// on whatever the current kernel state is
void
kerneltrap()
{
    int which_dev = 0;
    uint64 sepc = r_sepc();
    uint64 sstatus = r_sstatus();
    uint64 scause = r_scause();

    if((sstatus & SSTATUS_SPP) == 0)
        panic("kerneltrap: not from supervisor mode");
    if(intr_get() != 0)
        panic("kerneltrap: interrupts enabled");

    if((which_dev = devintr()) == 0){
        // interrupt or trap from an unknown source
        printf("scause=0x%lx sepc=0x%lx stval=0x%lx\n", scause, r_sepc(), r_stval());
        panic("kerneltrap");
    }

    // give up the CPU if this is a timer interrupt.
    if(which_dev == 2 && myproc() != 0){
        yield();
    }

    // the yield() may have caused some traps to occur,
    // so restore trap registers for use by kernelvec.S's sepc instruction.
    w_sepc(sepc);
    w_sstatus(sstatus);
}
```

Design

- kernel/trap.c void usertrap()

user space에서

which_dev == 2 즉, timer interrupt가 발생하면
yield()를 실행 해 현재 실행 중인 process가 CPU를
포기하게 한다.

```
usertrap(void)
{
    if(r_scause() == 8){

        if(killed(p))
            exit(-1);

        // sepc points to the ecall instruction,
        // but we want to return to the next instruction.
        p->trapframe->epc += 4;

        // an interrupt will change sepc, scause, and sstatus,
        // so enable only now that we're done with those registers.
        intr_on();

        syscall();
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
        printf("usertrap(): unexpected scause 0x%lx pid=%d\n", r_scause(), p->pid);
        printf("                sepc=0x%lx stval=0x%lx\n", r_sepc(), r_stval());
        setkilled(p);
    }

    if(killed(p))
        exit(-1);

    // give up the CPU if this is a timer interrupt.
    if(which_dev == 2)
        yield();

    usertrapret();
}
```

Design

- kernel/trap.c void clockintr()

100ms가 지날 때마다 ticks를 1 증가시킨다.

다음 timer interrupt가 100ms 뒤에 발생하도록 예약한다.

- kernel/trap.c int devintr()

interrupt를 판별하고 맞게 분배를 하는데, timer interrupt가 올 경우 clockintr() 실행 후 2를 return 해 which_dev 값을 정함

```
int
devintr()
{
    uint64 scause = r_scause();

    if(scause == 0x8000000000000009L){
        // this is a supervisor external interrupt.
        // ask for the next timer interrupt. this also clears
        // the interrupt request. 1000000 is about a tenth
        // of a second.
        w_stimecmp(r_time() + 1000000);

        // irq indicates which device interrupted.
        int irq = plic_claim();

        if(irq == UART0_IRQ){
            uartintr();
        } else if(irq == VIRTIO0_IRQ){
            virtio_disk_intr();
        } else if(irq){
            printf("unexpected interrupt irq=%d\n", irq);
        }

        // the PLIC allows each device to raise at most one
        // interrupt at a time; tell the PLIC the device is
        // now allowed to interrupt again.
        if(irq)
            plic_complete(irq);

        return 1;
    } else if(scause == 0x8000000000000005L){
        // timer interrupt.
        clockintr();
        return 2;
    } else {
        return 0;
    }
}
```

```
void
clockintr()
{
    if(cpuid() == 0){
        acquire(&tickslock);
        ticks++;
        wakeup(&ticks);
        release(&tickslock);
    }

    // ask for the next timer interrupt. this also clears
    // the interrupt request. 1000000 is about a tenth
    // of a second.
    w_stimecmp(r_time() + 1000000);
}
```

Design

- kernel/proc.c void scheduler(void)

앞 내용을 요약하자면,

timer interrupt가 1tick(100ms)마다 일어나고 그때마다 yield()를 실행해 실행 중인 process가 CPU를 포기하고 scheduler에서 다른 RUNNABLE한 process가 CPU를 선점하게 된다.

즉, xv6 스케줄러는 Round Robin policy가 적용되어 있다.

```
void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();

    c->proc = 0;
    for(;;){
        // The most recent process to run may have had interrupts
        // turned off; enable them to avoid a deadlock if all
        // processes are waiting.
        intr_on();

        int found = 0;
        for(p = proc; p < &proc[NPROC]; p++) {
            acquire(&p->lock);
            if(p->state == RUNNABLE) {
                // Switch to chosen process. It is the process's job
                // to release its lock and then reacquire it
                // before jumping back to us.
                p->state = RUNNING;
                c->proc = p;
                swtch(&c->context, &p->context);

                // Process is done running for now.
                // It should have changed its p->state before coming back.
                c->proc = 0;
                found = 1;
            }
            release(&p->lock);
        }

        if(found == 0) {
            // nothing to run; stop running on this core until an interrupt.
            intr_on();
            asm volatile("wfi");
        }
    }
}
```

github-classroom[bot], 2 weeks ago • Initial commit ...

Design

- kernel/main.c

Xv6가 처음 실행될 때 여러 값들에 대한 초기화를 한다.

이때 procinit(), userinit()에 집중을 했다.

```
void main()
{
    if(cpuid() == 0){
        consoleinit();
        printfinit();
        printf("\n");
        printf("xv6 kernel is booting\n");
        printf("\n");
        kinit();           // physical page allocator
        kvminit();         // create kernel page table
        kvmithart();       // turn on paging
        procinit();        // process table
        trapinit();        // trap vectors
        trapinithart();    // install kernel trap vector
        plicinit();        // set up interrupt controller
        plicinithart();    // ask PLIC for device interrupts
        binit();           // buffer cache
        iinit();           // inode table
        fileinit();        // file table
        virtio_disk_init(); // emulated hard disk

        initqueue();       // MLFQ, FCFS 큐 초기화

        userinit();        // first user process
        __sync_synchronize();
    }
}
```

Design

- kernel/proc.c

procinit()

- proc table의 process state를 모두 UNUSED로 바꾼다.

userinit()

- user/initcode.S에서 어셈블리로 정의된 코드initcode는 exec("/init")을 실행하게 한다.

```
// initialize the proc table.
void
procinit(void)
{
    struct proc *p;

    initlock(&pid_lock, "nextpid");
    initlock(&wait_lock, "wait_lock");
    for(p = proc; p < &proc[NPROC]; p++) {
        initlock(&p->lock, "proc");
        p->state = UNUSED;
        p->kstack = KSTACK((int) (p - proc));
    }
}
```

```
// Set up first user process.
void
userinit(void)
{
    struct proc *p;

    p = allocproc();
    initproc = p;

    // allocate one user page and copy initcode's instructions
    // and data into it.
    uvmfirst(p->pagetable, initcode, sizeof(initcode));
    p->sz = PGSIZE;

    // prepare for the very first "return" from kernel to user.
    p->trapframe->epc = 0;      // user program counter
    p->trapframe->sp = PGSIZE;  // user stack pointer

    safestrcpy(p->name, "initcode", sizeof(p->name));
    p->cwd = namei("/");

    p->state = RUNNABLE;

    release(&p->lock);
}
```

Design

- kernel/spinlock.c

void initlock(struct spinlock *lk, char *name)

- spinlock 구조체를 사용해 lock을 초기화한다.
- acquire(), release() 함수를 사용해 lock을 얻고 해제한다.

```
struct spinlock {
    uint locked;           // Is the lock held?

    // For debugging:
    char *name;            // Name of lock.
    struct cpu *cpu;       // The cpu holding the lock.
};
```

```
void
initlock(struct spinlock *lk, char *name)
{
    lk->name = name;
    lk->locked = 0;
    lk->cpu = 0;
}
```

```
acquire(&p->lock);
if(p->state == RUNNABLE) {
    // Switch to chosen process. It is the process's job
    // to release its lock and then reacquire it
    // before jumping back to us.
    p->state = RUNNING;
    c->proc = p;
    swtch(&c->context, &p->context);

    // Process is done running for now.
    // It should have changed its p->state before coming back.
    c->proc = 0;
    found = 1;
}
release(&p->lock);
```

Design

- user/init.c -> pid=1
 - exec("sh", argv); 로 셸 프로세스를 실행한다.
- user/sh.c -> pid=2
 - getcmd()에서 cmd를 읽고 해당 command를 실행한다.

결론적으로 kernel 영역의 proc table에서 pid 1은 init.c가 차지하고 있고 pid 2는 sh.c가 차지하고 있는 것이다.

임의로 유저 테스트 프로그램을 만들어서 실행하면 pid 3부터 시작하게 된다.

user/init.c

```
main(void)
{
    if(open("console", O_RDWR) < 0){
        mknod("console", CONSOLE, 0);
        open("console", O_RDWR);
    }
    dup(0); // stdout
    dup(0); // stderr

    for(;;){
        printf("init: starting sh\n");
        pid = fork();
        if(pid < 0){
            printf("init: fork failed\n");
            exit(1);
        }
        if(pid == 0){
            exec("sh", argv);
            printf("init: exec sh failed\n");
            exit(1);
        }
    }
}
```

user/sh.c

```
int
main(void)
{
    static char buf[100];
    int fd;

    // Ensure that three file descriptors are open.
    while((fd = open("console", O_RDWR)) >= 0){
        if(fd >= 3){
            close(fd);
            break;
        }
    }

    // Read and run input commands.
    while(getcmd(buf, sizeof(buf)) >= 0){
        if(buf[0] == 'c' && buf[1] == 'd' && buf[2] == ' '){
            // Chdir must be called by the parent, not the child.
            buf[strlen(buf)-1] = 0; // chop \n
            if(chdir(buf+3) < 0)
                fprintf(2, "cannot cd %s\n", buf+3);
            continue;
        }
    }
}
```

Design

- kernel/proc.c

sleep(void *chan, struct spinlock *lk)

- 조건 chan에 대해 기다린다.
- process state를 SLEEPING으로 바꾼 후 sched()로 CPU를 포기한다.

wakeup(void *chan)

- 조건 chan을 기다리는 모든 process를 깨운다.

```
void
sleep(void *chan, struct spinlock *lk)
{
    struct proc *p = myproc();

    // Must acquire p->lock in // Wake up all processes sleeping on chan.
    // change p->state and then // Must be called without any p->lock.
    // Once we hold p->lock, we void
    // guaranteed that we won't wakeup(void *chan)
    // (wakeup locks p->lock), {
    // so it's okay to release struct proc *p;

    acquire(&p->lock); //DOC: for(p = proc; p < &proc[NPROC]; p++) {
    release(lk);         if(p != myproc()){
                        acquire(&p->lock);
                        if(p->state == SLEEPING && p->chan == chan) {
                            p->state = RUNNABLE;
                            release(&p->lock);
                        }
                    }
}

    // Go to sleep.
    p->chan = chan;
    p->state = SLEEPING;

    sched();

    // Tidy up.
    p->chan = 0;

    // Reacquire original lock.
    release(&p->lock);
    acquire(lk);
}
```

Design

- 과제는 MLFQ mode scheduler, FCFS mode scheduler를 구현하는 것이다.
- 기존 scheduler에서 RUNNING할 프로세스 선택을 어떻게 하고 프로세스가 언제 yield 되는 지를 파악했고 이를 바탕으로 다음과 같이 scheduler 설계 구상을 했다.
- FCFS mode scheduler
 1. default scheduler와 동일하게 proc table을 돌면서 RUNNABLE한 process를 찾은 뒤 해당 process가 swch()를 통해 CPU를 선점한다.
 2. timer interrupt가 발생했을 때 FCFS 모드일 때는 yield()가 발생하지 않게 해 process가 CPU를 포기하지 못하게 한다.
- MLFQ mode scheduler
 1. L0, L1, L2 queue를 linked list로 구현한다. 이때 queue는 FIFO이다.
 2. L0부터 L2까지 차례대로 보면서 RUNNABLE한 process를 dequeue한다.
 3. 1tick마다 context switch가 발생하기 때문에 process가 yield를 실행해 CPU를 포기하고 scheduler에 돌아왔을 때 run tick을 1 증가시킨다.
 4. run tick과 time quantum을 바탕으로 해당 process의 queue level과 priority를 계산한다.
 5. queue level에 맞는 queue에 다시 enqueue한다.
 6. queue에 node를 추가하거나 삭제할 때 동기화를 위한 lock을 부여한다.

Implementation

- kernel/proc.h

current_mode: 현재 CPU의 scheduler mode를 나타낸다.

DEFAULT, MLFQ, FCFS 모드를 저장한다.

struct queue: 크기가 NPROC인 array로 queue를 만든다.
Head와 tail index 위치를 변수에 저장한다.
queue는 circular queue로 구현했다.

struct proc: MLFQ를 구현하기 위해 proces가 속한 큐 레벨(qlev),
우선순위(priority), 현재 레벨의 큐에서 사용한 시간
(run_ticks), 현재 레벨 큐의 time quantum(time_quantum)을
추가한다.

```
enum sched_mode { DEFAULT, MLFQ, FCFS }; // 스케줄러 모드
extern enum sched_mode current_mode;

...

struct queue { // 큐
    int head;
    int tail;
    struct proc *node[NPROC];
};
```

```
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state; // Process state
    void *chan; // If non-zero, sleeping on chan
    int killed; // If non-zero, have been killed
    int xstate; // Exit status to be returned to parent's wait
    int pid; // Process ID

    int qlev; // 큐 레벨
    int priority; // 우선순위
    int run_ticks; // 현재 레벨의 큐에서 사용한 시간
    int time_quantum; // time quantum = 2*qlev + 1 tick

    // wait_lock must be held when using this:
    struct proc *parent; // Parent process

    // these are private to the process, so p->lock need not be held.
    uint64 kstack; // Virtual address of kernel stack
    uint64 sz; // Size of process memory (bytes)
    pagetable_t pagetable; // User page table
    struct trapframe *trapframe; // data page for trampoline.S
    struct context context; // switch() here to run process
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd; // Current directory
    char name[16]; // Process name (debugging)
};
```

Implementation

- kernel/def.h

kernel/proc.h에서 새로 선언한 struct queue와
kernel/proc.c에서 새로 만든 function들을 def.h에 선언해준다.

- kernel/proc.c

NQUEUE: mlfq 레벨 개수 3개

BOOST_TICKS: priority_boost가 몇 tick마다 발생할 지 나타냄 50ticks

struct mlfq: 3개의 queue와 lock으로 구성됨. queue를 한 개 씩 lock으로 보호할 수도 있지만 CPU가 1개이기 때문에 구현하기 쉽도록 3개의 queue 전체에 대한 lock을 만들기 위해 구조체로 묶음.

current_mode = FCFS: default scheduler mode를 FCFS로 설정.

extern int ticks: trap.c에서 정의된 ticks를 proc.c에서도 접근 가능하게 함.

def.h

```
struct queue;
void      initqueue(void);
void      enqueue(struct queue *q, struct proc *p);
struct proc* dequeue(struct queue *q);
struct proc* dequeue_priority(struct queue *q);
// void      remove(struct queue *q, struct proc* p);
int      getlev(void);
int      setpriority(int pid, int priority);
int      mlfqmode(void);
int      fcfsmode(void);
void      priority_boost(void);
```

proc.c

```
#define NQUEUE 3 // MLFQ 레벨: 3
#define BOOST_TICKS 50 // global tick % 50 == 0 마다 Priority boosting

You, 6 minutes ago | 1 author (You)
struct {
    struct queue level[NQUEUE]; // L0, L1, L2 큐 선언
    struct spinlock lock;
} mlfq;
enum sched_mode current_mode = FCFS; // default 스케줄러 모드
extern uint ticks; //global tick
```

Implementation

- kernel/proc.c

void scheduler(void) – FCFS mode

- 기존 xv6는 scheduler()에서 proc table을 하나씩 순회하면서 RUNNABLE한 process를 선택하고 swtch()를 한다. 먼저 생성된 process는 pid 값이 작기에 이를 기준으로 creation time을 비교하면 된다. 하지만 proc table에서 index가 작다고 creation time이 우선임은 보장하지 않는다. 따라서 proc table을 돌면서 RUNNABLE한 process 중 pid가 가장 작은 process를 선택한다.
- 기존 scheduler는 Round Robin Policy가 적용되어 1 tick마다 yield()가 호출된다. 그래서 kernel/trap.c의 usertrap(), kerneltrap()에서 현재 scheduler mode가 FCFS면 yield()가 호출되지 않게 해 현재 CPU를 선점한 process가 time interrupt로 인한 CPU 포기가 발생하지 않도록 막는다.

trap.c

```
usertrap(void)
{
    // give up the CPU if this is a timer interrupt.
    if(which_dev == 2){
        if(current_mode != FCFS)
            yield();
    }

    usertrapret();
}
```

```
kerneltrap()
{
    // give up the CPU if this is a timer interrupt.
    if(which_dev == 2 && myproc() != 0){
        if(current_mode != FCFS)
            yield();
    }
}
```

proc.c

```
if(current_mode == FCFS){ // FCFS 모드
    struct proc *mp = 0;

    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == RUNNABLE) {
            if(mp == 0 || p->pid < mp->pid){
                if(mp) release(&mp->lock);
                mp = p;
                continue;
            }
        }
        release(&p->lock);
    }

    if(mp){
        mp->state = RUNNING;
        c->proc = mp;
        swtch(&c->context, &mp->context);
        c->proc = 0;
        found = 1;
        release(&mp->lock);
    }
}
```

Implementation

- kernel/proc.c

void scheduler(void) – MLFQ mode

- MLFQ mode의 경우 일단 처음에 global tick이 BOOST_TICKS(50) 배수면 priority boosting을 해준다.
- FCFS mode와 달리 proc table을 보지 않고 mlfq queue를 L0부터 L2까지 차례대로 보면서 RUNNABLE한 process를 찾는다. swtch 이후에 process가 CPU를 포기해 돌아오면 run_ticks를 1 증가한다.
- state가 RUNNABLE이라면 run_ticks와 time_quantum을 비교해 L0에서 L1으로 옮길지, L1에서 L2로 옮길지, L2라면 priority를 낮출지를 결정하고 run_ticks를 0으로 reset한다. 이후 queue level에 맞게 time quantum을 다시 설정한 뒤 해당 queue에 enqueue한다.

```
}else if(current_mode == MLFQ){ //MLFQ 모드
    if(ticks > 0 && ticks % BOOST_TICKS == 0){
        priority_boost();
    }
    for(int qlevel = 0; qlevel < NQUEUE; qlevel++){
        if(qlevel == NQUEUE - 1){
            p = dequeue_priority(&mlfq.level[qlevel]);
        }else {
            p = dequeue(&mlfq.level[qlevel]);
        }
        if(!p) continue;
        acquire(&p->lock);
        if(p->state == RUNNABLE){
            p->state = RUNNING;
            c->proc = p;
            swtch(&c->context, &p->context);
            c->proc = 0;
            p->run_ticks++;
            if(p->state == RUNNABLE){
                if(p->run_ticks >= p->time_quantum){
                    if(p->qlev < NQUEUE - 1){
                        p->qlev++;
                    }else if(p->qlev == NQUEUE - 1){
                        if(p->priority > 0)p->priority--;
                    }
                    p->run_ticks = 0;
                }
                p->time_quantum = 2 * p->qlev + 1;
                enqueue(&mlfq.level[p->qlev], p);
            }
            found = 1;
            release(&p->lock);
            break;
        }
        release(&p->lock);
    }
}
```

Implementation

- kernel/proc.c

void initqueue(void)

- MLFQ 3개를 초기화한다.
- kernel/main.c에 호출해 xv6가 시작할 때 초기화할 수 있게 한다.

proc.c

```
void
initqueue(void)
{
    for(int i=0; i<NQUEUE; i++){
        mlfq.level[i].head = 0;
        mlfq.level[i].tail = 0;
    }
    initlock(&mlfq.lock, "mlfq");
}
```

main.c

```
void
main()
{
    if(cpuid() == 0){
        consoleinit();
        printfinit();
        printf("\n");
        printf("xv6 kernel is booting\n");
        printf("\n");
        kinit();           // physical page allocator
        kvmalloc();        // create kernel page table
        kvmallocinit();    // turn on paging
        procinit();        // process table
        trapinit();        // trap vectors
        trapinit();        // install kernel trap vector
        plicinit();        // set up interrupt controller
        plicinit();        // ask PLIC for device interrupts
        binit();           // buffer cache
        iinit();           // inode table
        fileinit();        // file table
        virtio_disk_init(); // emulated hard disk

        initqueue();       // MLFQ, FCFS 큐 초기화

        userinit();        // first user process
        __sync_synchronize();
    }
```

Implementation

- kernel/proc.c

void priority_boost(void)

- proc table을 처음부터 돌면서 RUNNABLE하고 L0에 없는 process들은 L0로 enqueue한다.
- 해당 process의 qlevel, run_ticks, priority, time_quantum을 초기화한다. 이때, 위의 작업은 process의 lock을 건 채로 한다.
- mlfq에 lock을 걸고 L1, L2 queue를 초기화한다.

```
void
priority_boost(void){
    for(struct proc *p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->state == RUNNABLE && p->qlev != 0){
            enqueue(&mlfq.level[0], p);
        }
        p->qlev = 0;
        p->run_ticks = 0;
        p->priority = 3;
        p->time_quantum = 2 * p->qlev + 1;
        release(&p->lock);
    }

    acquire(&mlfq.lock);
    for(int qlevel = 1; qlevel < NQUEUE; qlevel++){
        mlfq.level[qlevel].head = 0;
        mlfq.level[qlevel].tail = 0;
    }
    release(&mlfq.lock);
}
```

Implementation

- kernel/proc.c

void wakeup(void *chan)

- wakeup()은 proc table을 돌면서 SLEEPING 상태면서 chan 조건을 성립하는 process를 전부 RUNNABLE 상태로 깨운다.
- MLFQ mode의 경우 scheduler가 접근할 수 있게 process의 priority, qlev, run_ticks, time_quantum을 초기화한 뒤 L0 queue에 process를 enqueue한다.

```
void
wakeup(void *chan)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++) {
        if(p != myproc()){
            acquire(&p->lock);
            if(p->state == SLEEPING && p->chan == chan) {
                p->state = RUNNABLE;
                if(current_mode == MLFQ){
                    p->priority = 3;
                    p->qlev = 0;
                    p->run_ticks = 0;
                    p->time_quantum = 2 * p->qlev + 1;
                    enqueue(&mlfq[p->qlev], p);
                }
            }
            release(&p->lock);
        }
    }
}
```

Implementation

- kernel/proc.c

int kill(int pid)

- kill()은 해당 pid의 process가 trap에 진입했을 때 종료될 수 있도록 killed를 1로 바꾸는데, 이때 process가 SLEEPING이면 RUNNABLE로 바뀌어서 scheduler가 해당 process를 찾을 수 있게 해 결과적으로 죽을 수 있게 만든다.
- 따라서, MLFQ mode의 경우 SLEEPING인 process를 RUNNABLE로 바꿀 때 L0 queue에 enqueue한다.

```
int
kill(int pid)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->pid == pid){
            p->killed = 1;
            if(p->state == SLEEPING){
                // Wake process from sleep().
                p->state = RUNNABLE;
                if(current_mode == MLFQ){
                    p->priority = 3;
                    p->qlev = 0;
                    p->run_ticks = 0;
                    p->time_quantum = 2 * p->qlev + 1;
                    enqueue(&mlfq.level[p->qlev], p);
                }
            }
            release(&p->lock);
            return 0;
        }
        release(&p->lock);
    }
    return -1;
}
```

Implementation

- kernel/proc.c

int fork(void)

- 새로운 process를 만들 때 process의 여러 값들을 초기화한다. 이때 MLFQ mode의 경우 L0 queue에 enqueue한다.

static void freeproc(struct proc *p)

- process를 UNUSED로 만들 때, 추가적으로 만들었던 qlev, priority, run_ticks, time_quantum을 초기화한다.

int fork(void)

```
acquire(&np->lock);
np->state = RUNNABLE;
np->run_ticks = 0;
if(current_mode == MLFQ){ // scheduling mode에 맞는 queue에 enqueue
    // 추가적인 np 초기화
    np->qlev = 0;
    np->priority = 3;
    np->time_quantum = 2*np->qlev + 1;
    enqueue(&mlfq.level[0], np);
}else if(current_mode == FCFS){
    // 추가적인 np 초기화
    np->qlev = -1;
    np->priority = -1;
    np->time_quantum = -1;
}
release(&np->lock);
return pid;
```

```
static void
freeproc(struct proc *p)
{
    if(p->trapframe)
        kfree((void*)p->trapframe);
    p->trapframe = 0;
    if(p->pagetable)
        proc_freepagetable(p->pagetable, p->sz);
    p->pagetable = 0;
    p->sz = 0;
    p->pid = 0;
    p->parent = 0;
    p->name[0] = 0;
    p->chan = 0;
    p->killed = 0;
    p->xstate = 0;
    p->state = UNUSED;

    p->qlev = 0;
    p->priority = 0;
    p->run_ticks = 0;
    p->time_quantum = 0;
}
```

Implementation

- kernel/proc.c

void enqueue(struct queue *q, struct proc *p)

- queue에 process를 넣는다.
- 넣을 proces가 null이거나 RUNNABLE이 아니라면 return 한다.
- mlfq 동기화를 위해 lock을 건다.
- 이미 해당 queue에 동일한 process가 있다면 추가하지 않는다.
- circular queue 구현을 위해 index를 queue size인 NPROC으로 나눈 나머지로 계산해 queue에 접근한다.
- queue가 꽉 찰 경우 L0, L1, L2 queue의 process들을 출력하고 panic을 출력한다.
- queue의 tail에 p를 추가하고 lock을 해제한다.

```
void
enqueue(struct queue *q, struct proc *p)
{
    if (!p || p->state != RUNNABLE)
        return;

    acquire(&mlfq.lock);

    // 중복 방지
    for (int i = q->head; i != q->tail; i = (i + 1) % NPROC) {
        if (q->node[i] == p){
            release(&mlfq.lock);
            return; // 이미 큐에 있음, 추가 안 함
        }
    }

    int next_tail = (q->tail + 1) % NPROC;
    if(next_tail == q->head){
        print_queue();
        panic("enqueue: queue full");
    }

    q->node[q->tail] = p;
    q->tail = next_tail;
    release(&mlfq.lock);
}
```

Implementation

- kernel/proc.c

struct proc* dequeue(struct queue *q)

- queue에 head에 있는 process를 제거하고 return한다.
- 동기화를 위해 mlfq에 lock을 건다.
- head부터 process를 하나씩 보면서 null process거나 RUNNABLE state가 아닌 process는 제거한다.
- RUNNABLE한 process를 보면 return한다.

```
struct proc*
dequeue(struct queue *q)
{
    acquire(&mlfq.lock);

    while(q->head != q->tail){
        struct proc *p = q->node[q->head];
        q->head = (q->head + 1) % NPROC;
        if (!p || p->state != RUNNABLE) {
            continue;
        }

        if(p->state == RUNNABLE){
            release(&mlfq.lock);
            return p;
        }
    }

    release(&mlfq.lock);
    return 0;
}
```

Implementation

- kernel/proc.c

struct proc* dequeue_priority(struct queue *q)

- L2는 priority가 높은 process를 우선으로 실행해야 하기 때문에, priority가 가장 높은 process를 제거하고 return 한다.
- priority queue 구현을 위한 여러 방법이 있지만 그 중 가장 단순한 방법을 사용했다.
- 동기화를 위해 mlfq에 lock을 건다.
- queue에 process가 없으면 return한다.
- RUNNABLE한 process들 중 priority가 가장 높은 process와 그 process의 위치를 찾는다.

```
struct proc*
dequeue_priority(struct queue *q)
{
    acquire(&mlfq.lock);

    int count = (q->tail - q->head + NPROC) % NPROC; // queue node 개수
    if(count == 0){
        release(&mlfq.lock);
        return 0;
    }

    int best_idx = -1;
    struct proc *best_proc = 0;

    int i = q->head;
    for(int j = 0; j < count; j++){ // priority가 가장 높은 node 찾기
        struct proc *p = q->node[i];
        if(p && p->state == RUNNABLE){
            if(!best_proc || p->priority > best_proc->priority){
                best_proc = p;
                best_idx = i;
            }
        }
        i = (i + 1) % NPROC;
    }

    if(!best_proc){
        release(&mlfq.lock);
        return 0;
    }
}
```

Implementation

- kernel/proc.c

struct proc* dequeue_priority(struct queue *q)

- 새로운 process array를 만들어서 전에 선택한 process가 없는 새로운 process array를 만든다.
- 기존 queue에 새로 만든 process array를 복사한다. 이로 인해 선택한 process를 queue에서 제거한다.
- 이후 선택한 process를 return한다.

```
struct proc *tmp[NPROC];
int new_tail = 0;

i = q->head;
for(int j = 0; j < count; j++){ // 선택된 node가 없는 새로운 queue 생성
    struct proc *p = q->node[i];
    if(i != best_idx){
        tmp[new_tail++] = p;
    }
    i = (i + 1) % NPROC;
}

q->head = 0;
q->tail = new_tail;
for(int k = 0; k < new_tail; k++){ // 기존 queue에 복사
    q->node[k] = tmp[k];
}

release(&mlfq.lock);
return best_proc;
}
```

Implementation

- kernel/proc.c

int mlfqmode(void)

- current_mode를 MLFQ로 바꾼다.
- MLFQ mode에서 실행하면 -1을 return한다.
- proc table을 돌면서 RUNNABLE인 process를 모두 L0 queue에 enqueue한다.
- global ticks를 0으로 초기화한다.
- 현재 process가 RUNNING 중이면 RUNNABLE로 state를 바꾸고 L0 queue에 enqueue한다. 이후 yield() 선언을 해 CPU를 양보한다. 이렇게 하지 않으면 mlfqmode()를 실행한 process가 종료될 때 L0 queue에 들어가지 않게 되어 문제가 발생했다.

```
int // MLFQ 모드 전환
mlfqmode(void)
{
    if(current_mode == MLFQ){
        return -1;
    }
    current_mode = MLFQ;
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++) {
        if(p->state != RUNNABLE) continue;
        acquire(&p->lock);
        p->qlev = 0;
        p->run_ticks = 0;
        p->priority = 3;
        p->time_quantum = 2 * p->qlev + 1;
        release(&p->lock);
        enqueue(&mlfq.level[0], p);
    }

    struct proc *cur = myproc();
    acquire(&cur->lock);
    if(cur->state == RUNNING){
        cur->state = RUNNABLE;
        cur->priority = 3;
        cur->qlev = 0;
        cur->time_quantum = 2 * cur->qlev + 1;
    }
    release(&cur->lock);

    acquire(&tickslock);
    ticks = 0;
    release(&tickslock);

    enqueue(&mlfq.level[0], cur);
    yield(); // CPU 양보
    return 0;
}
```

Implementation

- kernel/proc.c

int fcfsmode(void)

- current_mode를 FCFS로 바꾼다.
- FCFS mode에서 실행하면 -1을 return한다.
- proc table을 돌면서 모든 process의 priority, qlev, time_quantum을 초기화한다.
- mlfq를 초기화한다.
- global ticks를 0으로 초기화한다.

```
int // FCFS 모드 전환
fcfsmode(void)
{
    if(current_mode == FCFS){
        return -1;
    }
    current_mode = FCFS;

    struct proc *p;
    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        p->priority = -1;
        p->qlev = -1;
        p->time_quantum = -1;
        release(&p->lock);
    }

    acquire(&mlfq.lock);
    for(int i=0; i<NQUEUE; i++){
        mlfq.level[i].head = 0;
        mlfq.level[i].tail = 0;
    }
    release(&mlfq.lock);

    acquire(&tickslock);
    ticks = 0;
    release(&tickslock);
    return 0;
}
```

Implementation

- kernel/proc.c

void print_queue(void)

- L0, L1, L2에 process가 어떻게 위치하고 있는지 출력하는 함수다.
- 특별히 사용되진 않고 디버깅을 위한 코드이다.

```
void
print_queue(void)
{
    for(int lev = 0; lev < NQUEUE; lev++){
        int head = mlfq.level[lev].head;
        int tail = mlfq.level[lev].tail;
        if (head == tail) continue;

        printf("[L%d] ", lev);

        int idx = head;
        while (idx != tail) {
            struct proc *p = mlfq.level[lev].node[idx];
            if (p)
                printf("pid=%d,state=%d ", p->pid, p->state);
            else
                printf("null ");
            idx = (idx + 1) % NPROC;
        }
        printf("\n");
    }
}
```

Implementation

- kernel/proc.c – system call

int getlev(void)

- 현재 process의 queue level을 return한다. FCFS mode의 경우 99를 return한다.

int setpriority(int pid, int priority)

- proc table을 돌면서 process id가 pid인 process의 priority를 변경한다.
- invalid한 priority거나 pid인 process id가 없으면 각각 -2, -1을 return한다.

```
int // MLFQ에서 현재 큐의 level 반환
getlev(void)
{
    struct proc *p = myproc();
    if(current_mode == FCFS)
        return 99;
    return p->qlev;
}

int // 프로세스의 id가 pid인 프로세스의 priority를 변경
setpriority(int pid, int priority)
{
    if(priority < 0 || priority > 3){
        printf("Invalid priority!\n");
        return -2;
    }

    struct proc *p;
    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->pid == pid){
            p->priority = priority;
            release(&p->lock);
            return 0;
        }

        release(&p->lock);
    }
    printf("Process with the given pid doesn't exists!\n");
    return -1;
}
```

Implementation

- System call 구현

kernel/sysproc.c

```
uint64
sys_yield(void)
{
    yield();
    return 0;
}

uint64
sys_getlev(void)
{
    return getlev();
}

uint64
sys_setpriority(void)
{
    int pid, priority;
    argint(0, &pid);
    argint(1, &priority);
    return setpriority(pid, priority);
}

uint64
sys_mlfqmode(void)
{
    mlfqmode();
    return 0;
}

uint64
sys_fcfsmode(void)
{
    fcfsmode();
    return 0;
}
```

kernel/syscall.h

```
#define SYS_yield 22
#define SYS_getlev 23
#define SYS_setpriority 24
#define SYS_mlfqmode 25
#define SYS_fcfsmode 26
```

kernel/syscall.c

```
extern uint64 sys_yield(void);
extern uint64 sys_getlev(void);
extern uint64 sys_setpriority(void);
extern uint64 sys_mlfqmode(void);
extern uint64 sys_fcfsmode(void);

// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
    [SYS_fork]      sys_fork,
    [SYS_exit]      sys_exit,
    [SYS_wait]      sys_wait,
    [SYS_pipe]      sys_pipe,
    [SYS_read]      sys_read,
    [SYS_kill]      sys_kill,
    [SYS_exec]      sys_exec,
    [SYS_fstat]     sys_fstat,
    [SYS_chdir]     sys_chdir,
    [SYS_dup]       sys_dup,
    [SYS_getpid]    sys_getpid,
    [SYS_sbrk]      sys_sbrk,
    [SYS_sleep]     sys_sleep,
    [SYS_uptime]    sys_uptime,
    [SYS_open]      sys_open,
    [SYS_write]     sys_write,
    [SYS_mknod]     sys_mknod,
    [SYS_unlink]    sys_unlink,
    [SYS_link]      sys_link,
    [SYS_mkdir]     sys_mkdir,
    [SYS_close]     sys_close,
    [SYS_yield]     sys_yield,
    [SYS_getlev]    sys_getlev,
    [SYS_setpriority] sys_setpriority,
    [SYS_mlfqmode]  sys_mlfqmode,
    [SYS_fcfsmode]  sys_fcfsmode
}
```

user/user.h

```
void yield(void);
int getlev(void);
int setpriority(int pid, int priority);
int mlfqmode(void);
int fcfsmode(void);
```

user/usys.pl

```
entry("yield");
entry("getlev");
entry("setpriority");
entry("mlfqmode");
entry("fcfsmode");
```

Results

- xv6에 user/usertests.c를 mlfqmode에서 돌린 것이다.
- fcfs mode의 경우 자식 process가 무한루프를 도는 경우 테스트를 할 수 없기 때문에 테스트를 진행하지 않았다.
- 두 번의 usertests를 실행했을 때 모두 "ALL TESTS PASSED"가 출력됐다.

```
xv6 kernel is booting

init: starting sh
$ mlfqmode
$ usertests
usertests starting
test copyin: OK
test copyout: OK
test copyinstr1: OK
test copyinstr2: OK
test copyinstr3: OK
test rwsbrk: OK
test truncate1: OK
test truncate2: OK
test truncate3: OK
```

```
test pgbug: OK
test sbrkbugs: usertrap(): unexpected scause 0xc pid=6384
                sepc=0x4b0e stval=0x4b0e
usertrap(): unexpected scause 0xc pid=6385
                sepc=0x4b0e stval=0x4b0e

OK
test sbrklast: OK
test sbrk8000: OK
test badarg: OK
usertests slow tests starting
test bigdir: OK
test manywrites: OK
test badwrite: OK
test execout: OK
test diskfull: balloc: out of blocks
balloc: out of blocks
OK
test outofinodes: ialloc: no inodes
OK
ALL TESTS PASSED
$
```

```
test pgbug: OK
test sbrkbugs: usertrap(): unexpected scause 0xc pid=5594
                sepc=0x4b0e stval=0x4b0e
usertrap(): unexpected scause 0xc pid=5595
                sepc=0x4b0e stval=0x4b0e

OK
test sbrklast: OK
test sbrk8000: OK
test badarg: OK
usertests slow tests starting
test bigdir: OK
test manywrites: OK
test badwrite: OK
test execout: OK
test diskfull: balloc: out of blocks
balloc: out of blocks
OK
test outofinodes: ialloc: no inodes
OK
ALL TESTS PASSED
$
```

Results

- user/test1.c 파일을 만들어서 mlfq mode에서 큐 레벨이 잘 바뀌는지, priority boosting은 잘 되는지, Round Robin이 잘 되고 있는 지를 알아봤다.
- scheduler()에서 swtch() 이후 process의 run_time과 qlev, priority를 계산한 후 그 결과값을 출력했다.

```
swtch(&c->context, &p->context);
c->proc = 0;
p->run_ticks++;
if(p->state == RUNNABLE){
    if(p->run_ticks >= p->time_quantum){
        if(p->qlev < NQUEUE - 1){
            p->qlev++;
        }else if(p->qlev == NQUEUE - 1){
            if(p->priority > 0)p->priority--;
        }
        p->run_ticks = 0;
    }
    p->time_quantum = 2 * p->qlev + 1;
    printf("[tick=%d] pid=%d, state=%d, qlev=%d, priority=%d, run_ticks=%d, time_quantum=%d\n", ticks, p->pid, p->state, p->qlev, p->priority, p->run_ticks, p->time_quantum);
    enqueue(&mlfq.level[p->qlev], p);
}
found = 1;
release(&p->lock);
break;
}
release(&p->lock);
```

```
#include "kernel/types.h"
#include "user/user.h"
void busywork(int count) {
    volatile int x = 0;
    for (int i = 0; i < count * 100000000; i++) {
        x++;
    }
}
int main(void) {
    int pid = getpid();
    printf("[main] pid: %d\n", pid);
    printf("[main] 현재 큐 레벨: %d\n", getlev());
    printf("[main] MLFQ 모드로 전환\n");
    mlfqmode();
    printf("[main] 계산 작업 시작 (큐 레벨 변화 유도)\n");
    busywork(10);
    printf("[main] 계산 후 큐 레벨: %d\n", getlev());
    printf("[main] setpriority(%d, 0) 호출\n", pid);
    int result = setpriority(pid, 0);
    printf("[main] setpriority 결과: %d\n", result);
    printf("[main] 자식 프로세스 2개 생성\n");
    for (int i = 0; i < 2; i++) {
        int child = fork();
        if (child == 0) {
            if (i == 0) {
                printf("[child-%d] 짧은 작업 시작\n", i);
                busywork(5);
            } else {
                printf("[child-%d] 긴 작업 시작\n", i);
                busywork(15);
            }
            printf("[child-%d] 작업 종료, 큐 레벨: %d\n", i, getlev());
            exit(0);
        }
    }
    wait(0);
    wait(0);
    printf("[main] 모든 자식 종료. FCFS 모드 전환\n");
    fcfsmode();
    printf("[main] 테스트 종료\n");
    exit(0);
}
```

Results

```
xv6 kernel is booting
init: starting sh
$ test
[main] pid=3
[main] 현재 큐 레벨: 1
[main] MLFQ 모드로 전환
[main] 계산 작업 시작
[tick=1] pid=3, state=3, qlev=1, priority=3, run_ticks=0, time_quantum=3
[tick=2] pid=3, state=3, qlev=1, priority=3, run_ticks=1, time_quantum=3
[tick=3] pid=3, state=3, qlev=1, priority=3, run_ticks=2, time_quantum=3
[tick=4] pid=3, state=3, qlev=2, priority=3, run_ticks=0, time_quantum=5
[tick=5] pid=3, state=3, qlev=2, priority=3, run_ticks=1, time_quantum=5
[tick=6] pid=3, state=3, qlev=2, priority=3, run_ticks=2, time_quantum=5
[tick=7] pid=3, state=3, qlev=2, priority=3, run_ticks=3, time_quantum=5
[tick=8] pid=3, state=3, qlev=2, priority=3, run_ticks=4, time_quantum=5
[tick=9] pid=3, state=3, qlev=2, priority=2, run_ticks=0, time_quantum=5
[tick=10] pid=3, state=3, qlev=2, priority=2, run_ticks=1, time_quantum=5
[tick=11] pid=3, state=3, qlev=2, priority=2, run_ticks=2, time_quantum=5
[tick=12] pid=3, state=3, qlev=2, priority=2, run_ticks=3, time_quantum=5
[tick=13] pid=3, state=3, qlev=2, priority=2, run_ticks=4, time_quantum=5
[tick=14] pid=3, state=3, qlev=2, priority=1, run_ticks=0, time_quantum=5
[tick=15] pid=3, state=3, qlev=2, priority=1, run_ticks=1, time_quantum=5
[tick=16] pid=3, state=3, qlev=2, priority=1, run_ticks=2, time_quantum=5
[tick=17] pid=3, state=3, qlev=2, priority=1, run_ticks=3, time_quantum=5
[tick=18] pid=3, state=3, qlev=2, priority=1, run_ticks=4, time_quantum=5
[main] 계산 후 큐 레벨: 2
[main] setpriority(3, 0) 호출
[main] setpriority 결과: 0
[main] 자식 프로세스 2개 생성
[child-0] 짧은 작업 시작
[tick=20] pid=4, state=3, qlev=1, priority=3, run_ticks=0, time_quantum=3
[child-1] 긴 작업 시작
[tick=21] pid=5, state=3, qlev=1, priority=3, run_ticks=0, time_quantum=3
[tick=22] pid=4, state=3, qlev=1, priority=3, run_ticks=1, time_quantum=3
[tick=23] pid=5, state=3, qlev=1, priority=3, run_ticks=1, time_quantum=3
[tick=24] pid=4, state=3, qlev=1, priority=3, run_ticks=2, time_quantum=3
[tick=25] pid=5, state=3, qlev=1, priority=3, run_ticks=2, time_quantum=3
[tick=26] pid=4, state=3, qlev=1, priority=3, run_ticks=3, time_quantum=3
[tick=27] pid=5, state=3, qlev=1, priority=3, run_ticks=3, time_quantum=3
[tick=28] pid=4, state=3, qlev=1, priority=3, run_ticks=4, time_quantum=3
[tick=29] pid=5, state=3, qlev=2, priority=3, run_ticks=1, time_quantum=5
[tick=30] pid=4, state=3, qlev=2, priority=3, run_ticks=2, time_quantum=5
[tick=31] pid=5, state=3, qlev=2, priority=3, run_ticks=2, time_quantum=5
[tick=32] pid=4, state=3, qlev=2, priority=3, run_ticks=3, time_quantum=5
[tick=33] pid=5, state=3, qlev=2, priority=3, run_ticks=3, time_quantum=5
[tick=34] pid=4, state=3, qlev=2, priority=3, run_ticks=4, time_quantum=5
[tick=35] pid=5, state=3, qlev=2, priority=3, run_ticks=4, time_quantum=5
[tick=36] pid=4, state=3, qlev=2, priority=2, run_ticks=0, time_quantum=5
[tick=37] pid=5, state=3, qlev=2, priority=2, run_ticks=0, time_quantum=5
[child-0] 작업 종료, 큐 레벨: 2
[tick=38] pid=5, state=3, qlev=2, priority=2, run_ticks=1, time_quantum=5
[tick=39] pid=5, state=3, qlev=2, priority=2, run_ticks=2, time_quantum=5
[tick=40] pid=5, state=3, qlev=2, priority=2, run_ticks=3, time_quantum=5
[tick=41] pid=5, state=3, qlev=2, priority=2, run_ticks=4, time_quantum=5
[tick=42] pid=5, state=3, qlev=2, priority=1, run_ticks=0, time_quantum=5
[tick=43] pid=5, state=3, qlev=2, priority=1, run_ticks=1, time_quantum=5
[tick=44] pid=5, state=3, qlev=2, priority=1, run_ticks=2, time_quantum=5
[tick=45] pid=5, state=3, qlev=2, priority=1, run_ticks=3, time_quantum=5
[tick=46] pid=5, state=3, qlev=2, priority=1, run_ticks=4, time_quantum=5
[tick=47] pid=5, state=3, qlev=2, priority=0, run_ticks=0, time_quantum=5
[tick=48] pid=5, state=3, qlev=2, priority=0, run_ticks=1, time_quantum=5
[tick=49] pid=5, state=3, qlev=2, priority=0, run_ticks=2, time_quantum=5
[tick=50] pid=5, state=3, qlev=2, priority=0, run_ticks=3, time_quantum=5
[tick=51] pid=5, state=3, qlev=1, priority=3, run_ticks=0, time_quantum=3
[tick=52] pid=5, state=3, qlev=1, priority=3, run_ticks=1, time_quantum=3
[tick=53] pid=5, state=3, qlev=1, priority=3, run_ticks=2, time_quantum=3
[tick=54] pid=5, state=3, qlev=2, priority=3, run_ticks=0, time_quantum=5
[tick=55] pid=5, state=3, qlev=2, priority=3, run_ticks=1, time_quantum=5
[tick=56] pid=5, state=3, qlev=2, priority=3, run_ticks=2, time_quantum=5
[tick=57] pid=5, state=3, qlev=2, priority=3, run_ticks=3, time_quantum=5
[child-1] 작업 종료, 큐 레벨: 2
[main] 모든 자식 종료. FCFS 모드 전환
[main] 테스트 종료
$
```

run_ticks이 1 증가해 3이 될 때,
run_ticks >= time_quantum을 만족하므로, queue level
이 1 증가하고 time_quantum이 5로 바뀐다.

qlev이 2일 때 run_tick이 5가 되면 time_quantum과
같아지기 때문에 priority를 1 감소

출력 시점 때문에 오해가 있을 수 있지만,
50tick일 때, pid=5 process가 yield()로 CPU를 포기 ->
swtch() 아래부터 시작해 [tick=50]~ 로그 출력 ->
scheduler() 제일 위로 다시 올라와서 priority_boost() 실행
(이 때 qlev=0, run_ticks=0) -> pid=5 process 다시 선택 -
> 51tick일 때, pid=5 process가 yield()로 CPU를 포기 ->
swtch() 아래에서 run_ticks + 1 하지만, time_quantum과 같
기 때문에 qlev = 1, run_ticks = 0, time_quantum=3으로
변경 후 [tick=51]~ 로그 출력

Results

- Project01_Test의 test.c 파일을 연속으로 수행하기 위해 마지막 줄에 fcfsmode()로 변환하는 코드를 추가했다.
- 또한 priority_boostin이 일어나는지 확인하기 위해 kernel/proc.c에서 priority_boosting이 일어날 때 priority_boosting이 일어났다고 출력하고, 그 때 mlfq의 process를 출력하는 코드를 추가했다.

test.c

```
int main(int argc, char *argv[])
{
    // [Test 2] MLFQ test
    printf("\n[Test 2] MLFQ Scheduling\n");
    pid = fork_children();

    if (pid != parent)
    {
        for (i = 0; i < NUM_LOOP; i++)
        {
            int x = getlev();
            if (x < 0 || x >= MAX_LEVEL)
            {
                printf("Wrong level: %d\n", x);
                exit(1);
            }
            count[x]++;
        }

        printf("Process %d (MLFQ L0-L2 hit count):\n", pid);
        for (i = 0; i < MAX_LEVEL; i++)
            printf("L%d: %d\n", i, count[i]);
    }
    exit_children();

    printf("[Test 2] MLFQ Test Finished\n");
    printf("\nFCFS & MLFQ test completed!\n");
    fcfsmode();
    exit(0);
}
```

proc.c

```
void
priority_boost(void)
{
    printf("[priority_boost!]\n");
    for(struct proc *p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->state == RUNNABLE && p->qlev != 0){
            printf("[boost] pid=%d\n", p->pid);
            enqueue(&mlfq.level[0], p);
        }
        p->qlev = 0;
        p->run_ticks = 0;
        p->priority = 3;
        p->time_quantum = 2 * p->qlev + 1;
        release(&p->lock);
    }

    acquire(&mlfq.lock);
    for(int qllevel = 1; qllevel < NQUEUE; qllevel++){
        mlfq.level[qllevel].head = 0;
        mlfq.level[qllevel].tail = 0;
    }
    print_queue();
    release(&mlfq.lock);
}
```

Results

- Result Example에서는 mlfq scheduling test를 할 때 process 11, process 8, process 9, process 10이 실행되는데,
- 내 코드에서는 process 9, process 8, process 11, process 10 순서로 실행된다.
- 하지만 priority boosting도 제대로 되고 있고 mlfq scheduling 자체에는 문제가 없어보인다.
- L0, L1, L2에서 count의 비율도 대략 1:3:5의 비율을 유지하고 있다.

```
xv6 kernel is booting

init: starting sh
$ test
FCFS & MLFQ test start

[Test 1] FCFS Queue Execution Order
Process 4 executed 100000 times
Process 5 executed 100000 times
Process 6 executed 100000 times
Process 7 executed 100000 times
[Test 1] FCFS Test Finished

successfully changed to FCFS mode!
successfully changed to MLFQ mode!

[Test 2] MLFQ Scheduling
[priority_boost!]
[boost] pid=8
[boost] pid=9
[boost] pid=10
[boost] pid=11
[L0] pid=8,state=3 pid=9,state=3 pid=10,state=3 pid=11,state=3
Process 9 (MLFQ L0-L2 hit count):
L0: 12567
L1: 32866
L2: 54567
Process 8 (MLFQ L0-L2 hit count):
L0: 7428
L1: 35884
L2: 56688
Process 11 (MLFQ L0-L2 hit count):
L0: 12024
L1: 35422
L2: 52554
Process 10 (MLFQ L0-L2 hit count):
L0: 12069
L1: 30564
L2: 57367
[Test 2] MLFQ Test Finished

FCFS & MLFQ test completed!
$ test
```

```
$ test
FCFS & MLFQ test start

[Test 1] FCFS Queue Execution Order
Process 13 executed 100000 times
Process 14 executed 100000 times
Process 15 executed 100000 times
Process 16 executed 100000 times
[Test 1] FCFS Test Finished

successfully changed to FCFS mode!
successfully changed to MLFQ mode!

[Test 2] MLFQ Scheduling
[priority_boost!]
[boost] pid=17
[boost] pid=18
[boost] pid=19
[boost] pid=20
[L0] pid=17,state=3 pid=18,state=3 pid=19,state=3 pid=20,state=3
Process 18 (MLFQ L0-L2 hit count):
L0: 12574
L1: 29633
L2: 57793
Process 17 (MLFQ L0-L2 hit count):
L0: 9658
L1: 33551
L2: 56791
Process 20 (MLFQ L0-L2 hit count):
L0: 12874
L1: 36896
L2: 50230
Process 19 (MLFQ L0-L2 hit count):
L0: 12770
L1: 29197
L2: 58033
[Test 2] MLFQ Test Finished

FCFS & MLFQ test completed!
$
```

Troubleshooting

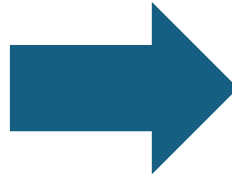
- scheduler의 테스트를 위해 기존 xv6에 포함된 user/usertests.c 파일을 실행했다.
- fcfs mode의 경우 killstatus()에서 멈추게 되는데, 자식 process가 무한루프에 빠질 경우 fcfs때문에 부모 process가 kill()을 호출할 수 없게 된다. 그래서 테스트를 하는 것이 무의미하다 생각했다.
- mlfq mode의 경우 execout() 테스트에서 에러가 나는 것을 확인했다. 원인을 분석한 결과 다음과 같은 문제가 있었다. 내 mlfq는 linked list를 통해 queue를 구현해서 그곳에 RUNNABLE한 process들을 넣었다. execout()의 경우 의도적으로 memory를 꽉 채우게 되는데, scheduler에서 RUNNABLE한 process를 dequeue하고, swtch()로 CPU를 선점한 뒤 나중에 yield()가 발생해 process가 CPU를 포기하고 enqueue되어야 할 때 memory가 꽉 차게 되면, enqueue를 실패하게 된다. 그러면 RUNNABLE한 process가 없어지고 kalloc() 실패로 인한 다른 panic이 발생하게 된다.
- 문제 해결을 위한 여러 방안이 있겠지만, 나는 그 중 linked-list로 구현한 queue를 array로 다시 구현하기로 했다. 다음 슬라이드에는 내가 수정한 코드들이 이어진다.

Troubleshooting

kernel/proc.h

```
enum sched_mode { DEFAULT, MLFQ, FCFS }; //스케줄러 모드
extern enum sched_mode current_mode;
You, yesterday | 1 author (You)
struct queue_node { // 큐 노드
    struct proc *p;
    struct queue_node *next;
};

You, yesterday | 1 author (You)
struct queue { // 큐
    struct queue_node *head;
    struct queue_node *tail;
};
```



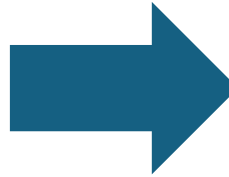
```
enum sched_mode { DEFAULT, MLFQ, FCFS }; //스케줄러 모드
extern enum sched_mode current_mode;
...
struct queue { // 큐
    int head;
    int tail;
    struct proc *node[NPROC];
};
```

Troubleshooting

kernel/def.h

```
11 struct queue;  
12 struct queue_node;
```

```
void initqueue(void);  
void enqueue(struct queue *q, struct proc *p);  
struct proc* dequeue(struct queue *q);  
struct proc* dequeue_priority(struct queue *q);  
void remove(struct queue *q, struct proc* p);  
int getlev(void);  
int setpriority(int pid, int priority);  
int mlfqmode(void);  
int fcfsmode(void);  
void priority_boost(void);
```



```
struct queue;  
void initqueue(void);  
void enqueue(struct queue *q, struct proc *p);  
struct proc* dequeue(struct queue *q);  
struct proc* dequeue_priority(struct queue *q);  
// void remove(struct queue *q, struct proc* p);  
int getlev(void);  
int setpriority(int pid, int priority);  
int mlfqmode(void);  
int fcfsmode(void);  
void priority_boost(void);
```

Troubleshooting

kernel/proc.c enqueue()

```
void
enqueue(struct queue *q, struct proc *p)
{
    if(p == 0){
        printf("enqueue: NULL process pointer\n");
        return;
    }

    acquire(&mlfq.lock);

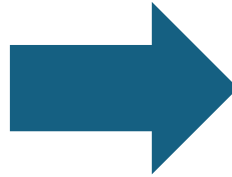
    // 중복 삽입 방지
    struct queue_node *cur = q->head;
    while(cur){
        if(cur->p == p){
            // 이미 큐에 존재
            release(&mlfq.lock);
            return;
        }
        cur = cur->next;
    }

    struct queue_node *node = (struct queue_node*)kalloc();
    if(!node){
        release(&mlfq.lock);
        panic("enqueue: kalloc failed");
    }

    node->p = p;
    node->next = 0;

    if(q->tail){
        q->tail->next = node;
    }else{
        q->head = node;
    }
    q->tail = node;

    release(&mlfq.lock);
}
```



```
void
enqueue(struct queue *q, struct proc *p)
{
    if (!p || p->state != RUNNABLE)
        return;

    acquire(&mlfq.lock);

    // 중복 방지
    for (int i = q->head; i != q->tail; i = (i + 1) % NPROC) {
        if (q->node[i] == p){
            release(&mlfq.lock);
            return; // 이미 큐에 있음, 추가 안 함
        }
    }

    int next_tail = (q->tail + 1) % NPROC;
    if(next_tail == q->head){
        print_queue();
        panic("enqueue: queue full");
    }

    q->node[q->tail] = p;
    q->tail = next_tail;
    release(&mlfq.lock);
}
```

Troubleshooting

kernel/proc.c dequeue()

```
struct proc*
dequeue(struct queue *q)
{
    acquire(&mlfq.lock);

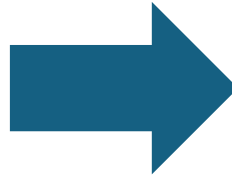
    while (q->head) {
        struct queue_node *node = q->head;
        struct proc *p = node->p;
        // 큐 정리
        q->head = node->next;
        if (!q->head)
            q->tail = 0;
        kfree((char*)node);

        // 포인터 유효성 검사
        if ((uint64)p < (uint64)proc || (uint64)p >= (uint64)(proc + NPROC)) {
            printf("dequeue: invalid pointer %p\n", p);
            continue; // 다음 노드로
        }

        // RUNNABLE 상태 확인
        if (p->state != RUNNABLE) {
            printf("dequeue: skipping non-RUNNABLE pid=%d state=%d\n", p->pid, p->state);
            continue; // 다음 노드로
        }

        // 정상적인 프로세스라면 반환
        release(&mlfq.lock);
        return p;
    }

    // 큐가 비어있다면 0 반환
    release(&mlfq.lock);
    return 0;
}
```



```
struct proc*
dequeue(struct queue *q)
{
    acquire(&mlfq.lock);

    while(q->head != q->tail){
        struct proc *p = q->node[q->head];
        q->head = (q->head + 1) % NPROC;
        if (!p || p->state != RUNNABLE) {
            continue;
        }

        if(p->state == RUNNABLE){
            release(&mlfq.lock);
            return p;
        }
    }

    release(&mlfq.lock);
    return 0;
}
```

Troubleshooting

kernel/proc.c dequeue_priority()

```
struct proc*
dequeue_priority(struct queue *q)
{
    acquire(&mlfq.lock);

    struct proc *best_proc = 0;
    struct queue_node *best_node = 0;
    struct queue_node *best_prev = 0;

    struct queue_node *prev = 0;
    struct queue_node *cur = q->head;

    while (cur) {
        struct proc *p = cur->p;
        if (!best_proc || p->priority < best_proc->priority) {
            best_proc = p;
            best_node = cur;
            best_prev = prev;
        }
        prev = cur;
        cur = cur->next;
    }

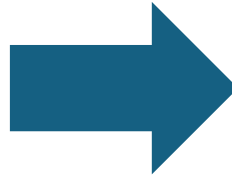
    if (best_node) {
        if (best_prev) best_prev->next = best_node->next;
        else q->head = best_node->next;

        if (best_node == q->tail) q->tail = best_prev;

        kfree((char*)best_node);

        release(&mlfq.lock);
        return best_proc;
    }

    release(&mlfq.lock);
    return 0;
}
```



```
struct proc*
dequeue_priority(struct queue *q)
{
    acquire(&mlfq.lock);

    int count = (q->tail - q->head + NPROC) % NPROC; // queue node 개수
    if(count == 0){
        release(&mlfq.lock);
        return 0;
    }

    int best_idx = -1;
    struct proc *best_proc = 0;

    int i = q->head;
    for(int j = 0; j < count; j++){ // priority가 가장 높은 node 찾기
        struct proc *p = q->node[i];
        if(p && p->state == RUNNABLE){
            if(!best_proc || p->priority > best_proc->priority){
                best_proc = p;
                best_idx = i;
            }
        }
        i = (i + 1) % NPROC;
    }

    if(!best_proc){
        release(&mlfq.lock);
        return 0;
    }

    struct proc *tmp[NPROC];
    int new_tail = 0;

    i = q->head;
    for(int j = 0; j < count; j++){ // 선택된 node가 없는 새로운 queue 생성
        struct proc *p = q->node[i];
        if(i != best_idx){
            tmp[new_tail++] = p;
        }
        i = (i + 1) % NPROC;
    }

    q->head = 0;
    q->tail = new_tail;
    for(int k = 0; k < new_tail; k++){ // 기존 queue에 복사
        q->node[k] = tmp[k];
    }

    release(&mlfq.lock);
    return best_proc;
}
```

Troubleshooting

kernel/proc.c remove()

```
void
remove(struct queue *q, struct proc* p)
{
    acquire(&mlfq.lock);

    struct queue_node *prev = 0;
    struct queue_node *cur = q->head;

    while (cur && cur->p != p) {
        prev = cur;
        cur = cur->next;
    }
    if(!cur){
        release(&mlfq.lock);
        return;
    }

    // Remove node
    struct queue_node *tmp = cur;
    if (prev) prev->next = cur->next;
    else q->head = cur->next;

    if (cur == q->tail) q->tail = prev;
    kfree((char*)tmp);

    release(&mlfq.lock);
    return;
}
```

